SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1023	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	U. Kavya	**Reg. No.:**	192472114
Date:	06-10-2026	**Duration:**	60 minutes

Code of Conduct

I U. Kavya(192472114)_____*(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: U. Kavya_____

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. **Understand the Problem Statement**
   - Explain the problem in your own words.
   - Identify the objectives of the proposed system.
   - State the scope of the problem.
2. **Identify Key Stakeholders and Challenges**
   - Identify all the stakeholders involved in the system.
   - Explain the roles and responsibilities of each stakeholder.
   - Discuss the major challenges and issues faced by the stakeholders.
3. **Analyze the Current Approach**
   - Describe the existing system or current process.
   - Identify its strengths and limitations.
   - Analyze the problems or gaps that need to be addressed.
4. **Propose a Suitable Solution Using Software Engineering Principles**
   - Design a software-based solution for the given problem.

- Describe the major functional modules of the proposed system.
- Identify the functional and non-functional requirements.
- Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.

5. **Justify the Chosen Methodology/Framework**
   - Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
   - Justify why the selected methodology/framework is suitable for the proposed system.
   - Explain how it supports successful project development and maintenance.
6. **Discuss Expected Outcomes and Limitations**
   - Describe the expected benefits of the proposed solution.
   - Explain how the solution addresses the identified challenges.
   - Discuss potential risks, implementation challenges, and limitations.
   - Suggest possible future enhancements.

Case Study Report: Container Security and Runtime Behaviour Monitoring Platform.

1. Understanding the Problem Statement

Containers (Docker images run on Kubernetes) make deployment fast and repeatable, but they also carry risk. An image can include vulnerable OS packages or libraries, hard-coded secrets, or a risky configuration such as running as root or in privileged mode. Even a clean image can be attacked at runtime through a cryptominer, a reverse shell, or a container escape. Most teams check security only once, if at all, so vulnerable images reach production and attacks go unnoticed. The task is to build one platform that secures the container from build to runtime.

Objectives

- Scan images for known vulnerabilities (CVEs), malware and exposed secrets.
- Validate configurations (Dockerfile, Kubernetes manifests) against security benchmarks.
- Monitor runtime behaviour such as processes, system calls, network traffic and file access.
- Detect suspicious or abnormal activity.
- Add automated security gates to the CI/CD pipeline.
- Support orchestration (Kubernetes) and centralized logging.
- Provide incident-response and audit mechanisms.

Scope

- In scope: image scanning, configuration validation, runtime monitoring, anomaly detection, CI/CD integration, centralized logs, alerting, dashboards, audit trail and automated response.
- Out of scope: host OS hardening, application business-logic testing, physical security, and network perimeter devices (firewalls, WAF).

2. Key Stakeholders and Challenges

Stakeholder	Role and responsibility	Major challenges
Developers	Write code, Dockerfiles and manifests	Security failures slow builds; scan reports are noisy; they lack security expertise

DevOps / Platform engineers	Maintain CI/CD pipelines and Kubernetes clusters	Adding gates without breaking delivery speed; managing policy across many clusters
Security team (SecOps / SOC)	Define policies, triage alerts, respond to incidents	Alert fatigue, false positives, no visibility into short-lived containers
Compliance / Audit officers	Verify adherence to standards (CIS, PCI-DSS, ISO 27001)	Missing evidence, manual audit effort
Project / Product managers	Plan delivery, budget and risk	Balancing security cost against release timelines
End users / customers	Use the hosted applications	Exposed to data breaches and downtime
Cloud / registry providers	Host registries, clusters and logs	Shared-responsibility ambiguity

Cross-cutting challenges

- Ephemeral containers (short-lived and numerous) are hard to trace.
- Third-party base images and dependencies are an unchecked supply chain.
- New CVEs appear daily, so an image that was safe at build time can become vulnerable later.
- Distinguishing normal behaviour from malicious behaviour is difficult.
- Security tools are fragmented, with separate tools for scanning, runtime and logging.

3. Analysis of the Current Approach

Existing process. In many organizations:

1. Developers build and push images with little or no scanning.
2. Security reviews are manual and happen late, just before release or after deployment.
3. Monitoring covers CPU and memory (e.g., Prometheus/Grafana), not security behaviour.
4. Logs stay scattered across nodes and are deleted when containers die.

Strengths

- Simple to set up, with low initial cost.
- Open-source tools such as Trivy, Falco and the ELK Stack already exist.
- Basic infrastructure monitoring gives some operational visibility.

Limitations and gaps

Gap	**Impact**
Security checks are late (not "shift-left")	Vulnerable images reach production
No enforced CI/CD gate	Policy depends on individuals
No runtime behaviour visibility	Attacks stay undetected for long periods
Isolated tools with no correlation	Slow investigation
Logs are lost when containers terminate	No forensic evidence
Manual response and audit	Delayed containment and compliance failures
No continuous re-scan of deployed images	New CVEs go unnoticed

4. Proposed Solution Using Software Engineering Principles

4.1 Solution overview

A **modular, event-driven platform** that spans three stages:

- **Build:** the CI/CD pipeline calls the Scanner and Policy Engine. A failed gate blocks the image from the registry.
- **Deploy:** a Kubernetes Admission Controller allows only signed images that have passed the scan.
- **Run:** lightweight runtime agents on each node stream events to the Anomaly Detection Engine. Alerts and logs go to the Dashboard, the SIEM and the Incident Response module.

4.2 Major functional modules

1. **Image Vulnerability Scanner:** scans OS packages, language dependencies and embedded secrets against CVE databases, and generates an SBOM (Software Bill of Materials).
2. **Configuration Validator:** checks Dockerfiles and Kubernetes manifests against CIS benchmarks (no root user, no privileged mode, resource limits, read-only filesystem).
3. **Policy Engine:** a central rule set (e.g., "block Critical CVEs", "allow only signed images") applied consistently across all stages.
4. **CI/CD Integration Module:** plugins for Jenkins and GitHub Actions that run scans and return pass/fail results.
5. **Admission Controller:** blocks non-compliant workloads at deploy time.
6. **Runtime Monitoring Agent:** a DaemonSet that observes system calls, process execution, network connections and file changes (eBPF-based).
7. **Anomaly Detection Engine:** compares behaviour with a learned baseline and with rules (e.g., a shell spawned in a web container, unexpected outbound connections).
8. **Centralized Logging and Analytics:** collects and stores logs and events (ELK/Loki) beyond container lifetime.
9. **Incident Response Module:** raises alerts (Slack, email, ticketing) and can quarantine pods, isolate the network or kill containers.
10. **Audit and Reporting Module:** tamper-proof audit trail, compliance reports and dashboards.
11. **Access Management:** role-based access for developers, security analysts and auditors.

4.3 Functional requirements

- FR1: Scan every image on build and periodically after deployment.
- FR2: Detect and rank vulnerabilities by severity (CVSS).
- FR3: Validate configuration against defined policies.
- FR4: Block pipeline or deployment when policy is violated.
- FR5: Monitor container processes, network and file activity in real time.
- FR6: Detect and classify anomalous behaviour.
- FR7: Send alerts and trigger automated response actions.
- FR8: Store searchable logs centrally.

- FR9: Provide dashboards and exportable audit reports.
- FR10: Support role-based access and policy management.

4.4 Non-functional requirements

- **Performance:** a scan should add under a few minutes to the pipeline, and agent overhead should be under about 5% CPU.
- **Scalability:** supports thousands of containers across multiple clusters.
- **Reliability and availability:** the platform should not become a single point of failure.
- **Security:** encrypted communication (TLS), secrets management, and least privilege.
- **Usability:** clear dashboards, actionable reports, minimal false positives.
- **Maintainability:** modular code, documented APIs, and automated updates of CVE data.
- **Interoperability:** works with Docker, Kubernetes, popular registries and SIEM tools.
- **Compliance:** aligned with CIS, NIST SP 800-190 and similar standards.

4.5 Software engineering principles applied

Principle	**How it is incorporated**
Modularity	Independent microservices (scanner, policy, detection, response) with well-defined REST/gRPC APIs; any module can be replaced without affecting the others
Scalability	Stateless services on Kubernetes with horizontal autoscaling; message queue (Kafka) to absorb event bursts
Maintainability	Separation of concerns, policy-as-code in version control, automated tests, CI/CD for the platform itself
Reliability	Redundant replicas, health checks, retries and graceful degradation (agents buffer events if the backend is down)
Usability	Role-specific dashboards, severity-based prioritization, remediation suggestions in scan reports
Security	Zero-trust design, RBAC, encrypted data, signed images and immutable audit logs; the platform's own components are scanned too

5. Justification of the Chosen Methodology

Chosen model: Agile (Scrum), combined with DevOps/DevSecOps practices.

Why it is suitable

1. **Evolving requirements:** threats, CVE sources and compliance rules change constantly. Agile's short sprints let the team adapt, whereas Waterfall's fixed requirements would go stale.
2. **Incremental delivery:** the platform can be released in useful increments (Sprint 1–2: image scanner and CI gate; Sprint 3–4: config validation and admission control; Sprint 5–6: runtime agent and logging; Sprint 7+: anomaly detection, response and audit).
3. **Early feedback:** developers and the security team test each increment and report false positives or usability problems early.
4. **Natural fit with the domain:** the product is itself a CI/CD security tool, so building it with DevOps (automated build, test, deploy and monitor) is consistent with its purpose.
5. **Risk management:** frequent reviews and retrospectives expose technical risks (e.g., eBPF performance) early. A Spiral model would manage risk well but is heavier and costlier for this team size.

How it supports development and maintenance

- The product backlog is continuously reprioritized (critical vulnerabilities first).
- Automated testing and CI keep quality high with every commit.
- Continuous deployment makes it easy to ship new detection rules and CVE feeds quickly.
- Retrospectives and monitoring feedback drive ongoing improvement after release.

6. Expected Outcomes and Limitations

Expected benefits

- Fewer vulnerable images reach production.
- Earlier detection of runtime threats (reduced time-to-detect and time-to-respond).
- Stronger CI/CD security through enforced gates.
- Full visibility of container workloads in one place.
- Faster, partly automated incident response.
- Audit-ready evidence that eases compliance.
- Secure, scalable deployments.

How challenges are addressed

Challenge	Solution
Late security checks	Shift-left scanning plus CI/CD gates
Ephemeral containers	Runtime agents and persistent central logs
Supply-chain risk	SBOM, image signing, trusted-registry policy
Alert fatigue	Severity scoring and baseline-driven detection
Tool fragmentation	One unified platform and policy engine
Manual audit/response	Automated reports and response playbooks

Risks, implementation challenges and limitations :

- **False positives and negatives:** anomaly detection needs tuning, and a baseline may take time to learn.
- **Performance overhead:** runtime monitoring adds some load on nodes.
- **Pipeline friction:** strict gates may slow developers if poorly tuned.
- **Coverage limits:** it cannot detect unknown (zero-day) exploits with certainty, and it depends on the quality of CVE databases.
- **Skill and cost:** the team needs Kubernetes and security expertise, and data storage costs grow.
- **Platform risk:** the platform itself becomes an attractive attack target and must be hardened.

Future enhancements:

- Machine-learning-based behavioural anomaly detection.
- Automated remediation, such as auto-patching base images and opening pull requests.
- Support for serverless and service-mesh environments.
- Multi-cloud and multi-cluster federation.

Rubrics for Calculation

Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder S Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution S Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/framework and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	**Mark**
Problem Understanding (30%)	/5
Stakeholder S Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution S Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25